

Bayesian based Re-parameterization for DNN Model Pruning

Xiaotong Lu*
xiaotonglu47@gmail.com
Xidian University
Xi'an, China

Gang Zhang
zhanggang03@baidu.com
Baidu Inc.
Beijing, China

Teng Xi*
xiteng01@baidu.com
Baidu Inc.
Beijing, China

Weisheng Dong
wsdong@mail.xidian.edu.cn
Xidian University
Xi'an, China

Baopu Li†
bpli.cuhk@gmail.com
Oracle Health and AI
Redwood City, USA

Guangming Shi
gmshi@xidian.edu.cn
Xidian University
Xi'an, China

ABSTRACT

Filter pruning, as an effective strategy to obtain efficient compact structures from over-parametric deep neural networks(DNN), has attracted a lot of attention. Previous pruning methods select channels for pruning by developing different criteria, yet little attention has been devoted to whether these criteria can represent correlations between channels. Meanwhile, most existing methods generally ignore the parameters being pruned and only perform additional training on the retained network to reduce accuracy loss. In this paper, we present a novel perspective of re-parametric pruning by Bayesian estimation. First, we estimate the probability distribution of different channels based on Bayesian estimation and indicate the importance of the channels by the discrepancy in the distribution before and after channel pruning. Second, to minimize the variation in distribution after pruning, we re-parameterize the pruned network based on the probability distribution to pursue optimal pruning. We evaluate our approach on popular datasets with some typical network architectures, and comprehensive experimental results validate that this method illustrates better performance compared to the state-of-the-art approaches.

CCS CONCEPTS

• **Computing methodologies** → *Artificial intelligence; Deep Neural Network Model Compression.*

KEYWORDS

Model Compression, Channel Pruning, Bayesian.

ACM Reference Format:

Xiaotong Lu, Teng Xi, Baopu Li, Gang Zhang, Weisheng Dong, and Guangming Shi. 2022. Bayesian based Reparameterization for DNN Model Pruning. In *Proceedings of the 30th ACM International Conference on Multimedia (MM '22, October 10–14, 2022, Lisboa, Portugal)*.

*Both authors contributed equally to this research. This work was done when Xiaotong Lu was an intern at Department of Computer Vision Technology (VIS), Baidu Inc. † means corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions.acm.org.
MM '22, October 10–14, 2022, Lisboa, Portugal.
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9203-7/22/10...\$15.00
<https://doi.org/10.1145/3503161.3547965>

'22), Oct. 10–14, 2022, Lisboa, Portugal. ACM, New York, NY, USA, 9 pages.
<https://doi.org/10.1145/3503161.3547965>

1 INTRODUCTION

Deep neural networks (DNN) has found many successful applications in various visual tasks such as image recognition, object detection and so on [10, 27, 32, 35]. Due to their over-parameterization, filter pruning[9, 22, 26], as an important tool for model acceleration and compression, has attracted increasingly more attention in recent years. The core idea of filter pruning is to remove unimportant or unworthy filters in redundant DNNs to get a compact representation of DNN with negligible performance loss (e.g., classification accuracy or detection accuracy). The filter pruning approach usually consists of four stages: pre-training of a DNN, measuring the importance of filters in DNN, removing unimportant filters according to a pre-defined criteria, and fine-tuning the pruned network.

Recent developments in filter pruning approaches have provided new insights that have led to many impressive improvements in pruning strategies, including methods that jointly pre-train and prune to improve efficiency [26], methods that search for a criterion that facilitates pruning for each layer [11], and methods that search in multiple dimensions (depth, width, kernel size, and resolution)[1]. However, filter pruning still faces some challenging problems, which lead to difficulties in realizing the full potential of the pruning method. First, different layers in filter pruning are generally treated as independent pruning process despite of the coupling between them. For example, for DNNs such as ResNet that contain continuous convolution layers, removing a layer in front of a network will result in a change in importance distribution of filters in the layers at the back end of the network. This dynamical evolution of filter importance has a subtle but important impact on the choice of pruning, while existing approaches to filter pruning tend to ignore this issue. In addition, the proportion of pruning per round in many related approaches is difficult to control. As a consequence of pruning, the lower the proportion of pruning per round and the more pruning rounds in a multi-round pruning process, the better the accuracy of the pruned model will be, but at a higher computational cost [18]; while the higher the proportion of pruning per round, the greater the damage to the parameters/distribution will cause, which will be accumulated round by round, resulting in irreversible accuracy loss. We believe that the loss in the pruning process is essentially due to the distribution change, therefore, we

aim to search for an optimal re-parameterization method that minimizes the distribution change before and after pruning to achieve the optimal/better pruning strategy.

To overcome the above two issues, we propose a new filter pruning paradigm for providing a more insightful analysis of the pruning from the perspective of probability distribution. Specifically, our approach is divided into two parts. First, we learn the distribution of each layer of the network based on Bayesian estimation, then the importance of channels can be described with the discrepancy of prior and posterior Bayesian distribution. Based on such a modelling process, we can effectively take into account continuous layers at the same time for channel pruning which are considered independently in most previous pruning methods. Then, we sample many networks and use them for Bayesian hyperparameter estimation. Furthermore, we use the learned distribution discrepancy to guide the re-parameterization of the pruned network that is based on channel reweighting and decay in order to maintain a consistent distribution of parameters in the network after pruning. And such a re-parameterization step can benefit our pruning method because it ensures that the change in distribution before and after each pruning round is minimized. Extensive comparison experiments on some benchmark datasets verify the superior performance of the proposed novel model pruning method. Contributions of this paper can be summarized as follows:

- A novel pruning framework based on Bayesian Estimation and re-parameterization is suggested to prune a network recursively, providing us a new perspective of model pruning.
- The pruning process is modelled from stochastic perspective, where Bayesian based sampling and prediction is leveraged to effectively guide the whole pruning process.
- Based on the obtained probability distribution, we reparameterize the pruned network via channel-level reweighting and decay, and show that such reparameterization help to achieve optimal parameterization, leading to more efficient pruning.

Comprehensive experiments verify the efficacy of our proposed method, and comparisons with other pruning methods shown that our method achieves the new state-of-the-art.

2 RELATED WORK

Filter Pruning. Filter pruning is structural pruning in the context of network compression, which requires measuring the importance of the filter. By removing unimportant channels based on the importance, filter pruning achieves the objective of network compression. The mission of filter pruning is to remove as many filters as possible without compromising the network performance. Specifically, recent work [14] decomposes the filter parameters into the product of sparsity and matrix, and prunes the filters globally with the help of group sparsity. Filter pruning via geometric median [13] measures the filter by deriving the concept of redundancy through geometric median values. In addition, [24] uses meta learning to automatically determine the possible pruned candidate networks and obtains the final pruned network through searching. Transformable architecture search [5] achieves superior performance using NAS and knowledge distillation to directly search the number of filters as well as the depth of the network. Joint search-and-training [26]

jointly implements the search and training process to achieve more than 2X speedup in speed. Most recent advances in this class include High-Rank Feature Map [22], Hinge between filter pruning and decomposition [19], Differentiable Markov Channel Pruning [9], Learned Global Ranking [4], structural redundancy reduction [34], Collaborative Compression [20] and methods for directly maximizing the performance of subnetworks [8]. Different from all the above pruning related works, we model the pruning process via Bayesian estimation and re-parameterization.

Neural Architecture Search. Neural architecture search (NAS) aims at finding the optimal network structure for different tasks. Early NAS methods [29, 31, 37] proposed to search structures using evolutionary algorithms or reinforcement learning. Recent advances attempt to reduce the burden of searching through gradient-based methods. DARTS [23] assigns different weights to possible operations and uses them to represent the probability of each candidate operation. DARTS learns these probabilities using a gradient descent algorithm to find the structure with the highest probability from the over-parameterized search space. ProxylessNAS [2] searches for architectures by sampling only the optimal two paths at each sampling to avoid time-consuming searches on large-scale vision tasks. OFA [1] trains a network that includes different architecture settings so it can be efficient for obtaining a specific subnetwork in a given deployment scenario. Our work is inspired by NAS but different from NAS since our goal aims to find the optimal subnetworks after pruning without searching process. Moreover, the optimal subnetwork is pruned through importance estimation via prior and posterior Bayesian distribution.

Bayesian-based network pruning. In [28], the author mainly uses the Bayesian model to remove some channels in each layer of the network to achieve pruning; Whereas in [25] the pruning of neurons and all their incoming and outgoing weights is achieved by Bayesian modeling using sparseness-inducing priors on hidden units (rather than individual weights). [36] introduced a new dropout-based redundancy measure to compute posterior hypothesis inter-layer dependencies, and achieved more optimal pruning by taking inter-layer dependencies into account in pruning. Although different, the application of these methods to the Bayesian model can be regarded as an optimization of a metric, and the corresponding output is the importance of the network weight, which will directly guide the pruning. Different from these methods, our Bayesian model is based on different rounds of iterative pruning, and its input and output are different from these methods. Second, unlike these methods, which can only optimize a part of the network, our framework (upper part in Figure 1) takes the overall network structure/accuracy as input and output, so it can be seen as an end-to-end optimization.

3 METHODOLOGY

In this section, we elaborate on the proposed method. First, we analyze the pruning operation by a Bayesian approach and propose a Bayesian-based pruning theory. Later, we propose a specific pruning method based on Bayesian estimation. Finally, we introduce the re-parameterization.

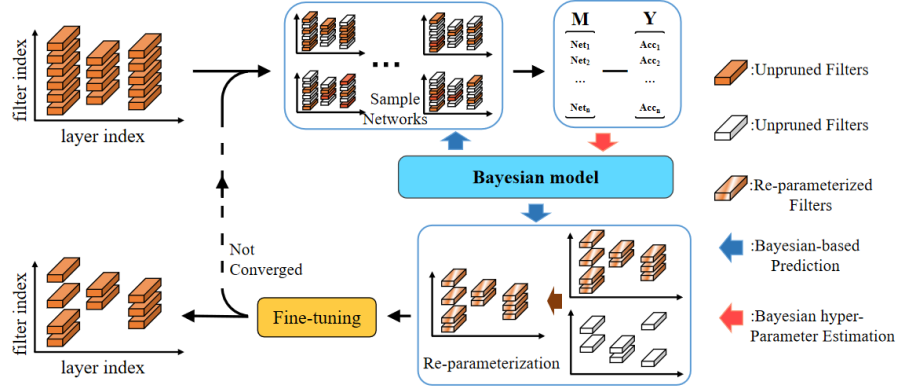


Figure 1: In our method, the pruned model is obtained by multiple iterations of pruning. In each iteration, we first obtain the structure of the network based on Bayesian prediction and sample it. Second, we update the Bayesian hyperparameters with the sampled structure-accuracy pairs. Finally, the optimal structure obtained is reparameterized and fine-tuned.

3.1 Preliminaries

We assume that a DNN $\mathcal{T}$ has L layers, where the filters at l^{th} layer can be represented as $\mathbf{W}^l = [w_1^l, w_2^l, \dots, w_{C^l}^l] \in \mathcal{R}^{C^{l-1} \times k^2 \times C^l}$, where C^l denotes the number of l^{th} layer's output channels, k is the spatial size of filters. The objective function of training can be formulated as:

$$\underset{\mathbf{W}}{\operatorname{argmin}} \ell_{ce}(y, f(x, \mathbf{W})) \quad (1)$$

where (x, y) denotes the input/label pair, $f(\cdot)$ represents the general mapping function of a DNN, ℓ_{ce} is the standard cross-entropy loss function. Defining the output of l^{th} layer in $\mathcal{T}$ as $O^l \in \mathcal{R}^{C^l \times b \times h \times w}$, then the computation in l^{th} layer can be represented as:

$$\begin{aligned} O_c^l &= w_c^l * O^{l-1}, \quad c = 1, 2, \dots, C^l \\ &= w_c^l * f^{l-1}(x, \mathbf{I}^{l-1}) \end{aligned} \quad (2)$$

where $*$ represents a convolution operator, $f^{l-1}(\cdot)$ denotes the mapping function corresponding to the $l-1$ layers of DNN and $\mathbf{I}^{l-1} \in \mathcal{R}^{C^{l-1}}$ represents the remained filters in the previous $l-1$ layers and $\mathbf{I} \in \mathcal{R}^L$ represents all remained filters. Note that the above formulation only represents the part of the convolutional computation where O is often activated as well, and for simplicity we assume that O is activated unless otherwise indicated.

After pruning, we assume the pruned network as $\hat{\mathcal{T}}$ with parameters $\hat{\mathbf{W}}^l = [\hat{w}_1^l, \hat{w}_2^l, \dots, \hat{w}_{\hat{C}^l}^l] (\hat{C}^l \leq C^l)$ and take the ratio of the pruned model's complexity and the original model's complexity as an explicit constraint. Then the objective function of pruning can be expressed as follows:

$$\hat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \ell_{ce}(y, f(x, \hat{\mathbf{W}})), \quad \frac{C(\hat{\mathbf{W}})}{C(\mathbf{W})} < r \quad (3)$$

where $C(\cdot)$ is the complexity of the DNN and r is a predefined pruning ratio, usually in terms of FLOPs and r for the pruning rate.

3.2 Modeling from Bayesian Perspective

The key issue in filter pruning is to evaluate as accurately as possible which filters in the network are redundant and to make the

pruning actions have as little impact as possible on the performance of the model. Some prevailing methods are to replace one-time pruning by multiple rounds of iterative pruning, since repeated pruning and training can retain a model's capacity compared to pruning a large number of parameters at once.

However, due to the nature of interconnection of neural network models, each filter pruned or not affects its subsequent convolutional layers, which leads to inappropriate pruning choices to be accumulated and amplified during iterative pruning process eventually leading to undesirable pruning results. Therefore, the ideal pruning method is to perform optimal pruning at each iteration, which is to enumerate all possible filter pruning combinations and select the most suitable one. But obviously, this method is so time-consuming that it is impossible to be applied to real scenarios. Therefore, we consider that optimal pruning is an asymptotic optimization method, where the pruning choices of adjacent layers are a priori and a posteriori, which is exactly suitable for Bayesian interpretation. Therefore, inspired by [6, 21], we propose for the first time to formulate the optimal pruning as a Joint Gaussian Process as:

$$P(\mathbf{Y} | \eta) = \frac{1}{\sqrt{2\pi}^n} \exp\left(-\frac{1}{2}(\mathbf{Y} - \eta\mathbf{M})\Sigma_{\mathbf{M}}^{-1}(\mathbf{Y} - \eta\mathbf{M})^T\right) \quad (4)$$

where $\mathbf{M} = [\mathbf{I}_1, \mathbf{I}_2, \dots, \mathbf{I}_n]^T$ represents the set of filter structures after pruning and $\mathbf{Y} = [y_1, y_2, \dots, y_n]^T$ is the ground truth vector of $\mathbf{M}$, and η is the hyperparameter to be estimated. Accordingly, $\eta\mathbf{M}$ represent the predicted accuracy of the corresponding pruned structure $\mathbf{M}$. Let $\eta \sim \mathcal{N}(\mu_\eta^t, \Sigma_\eta)$ represents the posterior distribution which depends on its prior distribution $\mathcal{N}(\mu_\eta^{t-1}, \Sigma_\eta)$. In this way, the optimal pruning problem is formulated as how to accurately estimate the hyperparameters after each round of pruning. Following the Theorem 1 in GP-NAS[21], the posterior update formulation for η after t rounds of iterations can be given as:

$$\mu_\eta^t = \mu_\eta^{t-1} + \mathbf{M}^T \Sigma_\eta (K + \mathbf{M}^T \Sigma_\eta \mathbf{Y})^{-1} \times (\mathbf{Y} - \mathbf{M} \mu_\eta^{t-1}) \quad (5)$$

where $\mathbf{M} \mu_\eta^{t-1}$ denotes the estimated accuracy of any pruned architecture $\mathbf{M}$ based on the prior distribution. We can see that the

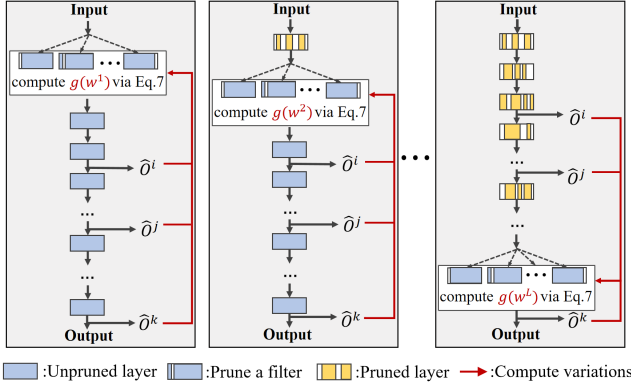


Figure 2: A pipeline that prunes the network once during iterative pruning of the network.

posterior distribution depends on both the sampled pruned architecture and its prior distribution. Different from GP-NAS that has only one distribution to be estimated, the distribution in each iteration for our problem here will slightly change, so we choose kernel function as standard squared exponential [30] kernel for easy converge.

3.3 Bayesian Estimation based Pruning

In this section, we specify an innovative metric: the filter feature representation function $g(\cdot)$, and since feature maps at multiple scales can jointly represent the feature distribution of a network [3] and shallow networks contain more global features [7], we select the first three different scales of feature maps $[O^i, O^j, O^k]$ for calculating $g(\cdot)$ (as shown in Figure 2). In order to better express the distribution properties of the network, we choose these three feature maps are determined at different locations of the model with different spatial resolutions and located before the downsampling operator. This choice is based on two consensuses: first, when the network structure changes, the corresponding featuremap will also change; Second, in the feature extraction network, the maximum loss of features comes from the downsampling operation. In combination with the block-based feature extraction network structure, the change in the feature extraction capability of the network can be measured by targeting the output feature map of each block, i.e., the feature map before the downsampling operation. We then define the feature representation function of O^i corresponding to w_c^l as the distance of the feature maps:

$$d_i(w_c^l) = \|O^i - \hat{O}^i\|_F^2 = \begin{cases} \|f^i(x, \mathbf{I}^i) - f^i(x, (\mathbf{I}^i - \mathbf{U}^{l-1} - w_c^l))\|_F^2 & l \leq i \\ \|f^i(x, \mathbf{I}^i) - f^i(x, (\mathbf{I}^i - \mathbf{U}^i))\|_F^2 & i < l \end{cases} \quad (6)$$

where $\mathbf{U}^i$ represents the filters to be pruned in the i -layer, and $(\mathbf{I}^i - w_c^l)$ represents filters in first i layers except w_c^l . Then we can formulate the feature representation function as:

$$g(w_c^l) = d_i(w_c^l) + d_j(w_c^l) + d_k(w_c^l) \quad (7)$$

where $g(w_c^l)$ represents the variations in the three chosen feature maps after removing the channels that correspond to w_c^l . Here

Algorithm 1: Strategy S for pruning the l^{th} layer

- 1 **Input:** $\{p(w_1^l), p(w_2^l), \dots, p(w_c^l)\}$, a *threshold*
- 2 **Initialization:** $Sum = 0, U_l = \emptyset$
 - 1: Resorting $\{p(w_1^l), p(w_2^l), \dots, p(w_c^l)\}$ from smallest to largest as P
 - 2: **while** $Sum < threshold$ **do**
 - 3: Add the first value of P to Sum
 - 4: Move the first value from P to U_l
 - 5: **end while**
 - 6: Pruning the filters corresponding to $p(w_c^l)$ in U_l

we define the feature representation function of w_c^l as the sum of the variations over the three feature maps, according to which we argue that a larger $g(w_c^l)$ representation of the c^{th} filter at layer l has a greater impact on the global distribution of the network, and, correspondingly, a filter with a smaller feature representation function is more likely to be pruned without affecting the expression performance of the model. However, given the large variation in spatial resolution between feature maps, making it difficult to determine pruning directly using $g(w_c^l)$, we use the probability of pruning after *softmax*. After all $g(w_c^l)$ is obtained, the probability of the filters in each layer can be calculated as:

$$p(w_c^l) = \frac{\exp(\hat{g}(w_c^l))}{\sum_{k=1}^{C^l} \exp(\hat{g}(w_k^l))} \quad (8)$$

where $\hat{g}(w_c^l)$ is the normalized $g(w_c^l)$:

$$\hat{g}(w_c^l) = \frac{g(w_c^l) - \text{mean}(g(w^l))}{\sqrt{\text{var}(g(w^l))}} \quad (9)$$

Here $p(w_1^l), p(w_2^l), \dots, p(w_c^l)$ can then represent the variations in the distribution of model features after a single filter in layer l is pruned: a larger $p(w_c^l)$ means that pruning this filter has a greater impact on the distribution of the model and thus it plays a more significant role in the model; conversely, it means that the presence of this filter has little impact on the model as a whole. Accordingly, when we prune the filters, we need to select the filters to be pruned layer by layer according to the policy S and update them to the pruned filter set $\mathbf{U}$. Here, S represents the strategy for determining the filters to be pruned at each pruning stage given an approximate pruning ratio, and this number varies according to the distribution of $p(w_c^l)$: to reduce the size fluctuations of the pruning model, we dynamically decide how many filters to prune based on the normalized sum of $p(w_c^l)$ rather than on the total number of filters as in Algorithm 1.

For a pruned network, we define its feature representation function under policy S as:

$$\ell_{feature} = d_i(\mathbf{U} | S) + d_j(\mathbf{U} | S) + d_k(\mathbf{U} | S) \quad (10)$$

where $\mathbf{U} | S$ represents the filter pruned under strategy S , and $d_i, j, k(\mathbf{U} | S)$, similar to Eq.6, represents the variation in the feature maps $[O^i, O^j, O^k]$ after pruning, respectively.

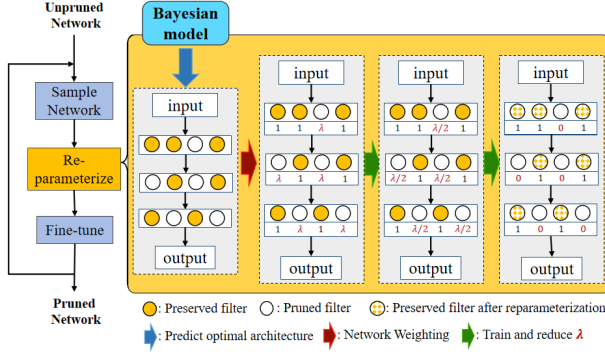


Figure 3: In the reparameterization, we first obtain the optimal structure of the Bayesian prediction, then weight this network and impose constraints on the unpruned filters, and finally reducing the weights to zero during training.

Then according to the definition of $d(\mathbf{U} | S)$, if $\ell_{feature}$ is small enough, it means that the overall distribution of the network does not change much, and there is not much loss for the feature distribution discrepancy between the pruned and unpruned networks after pruning, i.e., proper pruning. Therefore, in order not to destroy the features in the pruned network and to retain more information from the original network work, we adopt a cautious approach in designing the pruning strategy. Specifically, we set a very small pruning threshold (0.05 in our implementation) in each round of iterations and implement pruning. Finally, the desired pruning rate is achieved through several iteration rounds.

3.4 Reparameterization

After the pruning process based on Bayesian estimation, we can obtain a pruned network $\hat{\mathcal{T}}$ and a filter set $\mathbf{U}$ that should be pruned. Although the pruned network $\hat{\mathcal{T}}$ and the unpruned network $\mathcal{T}$ are close in the distribution of features, directly pruning the filter sets represented in $\mathbf{U}$ may still lead to irreversible errors, which will be accumulated in the next round of pruning. To mitigate such effects, we propose an optimal re-parameterization method to transform the feature distribution of the pruned network $\hat{\mathcal{T}}$.

In particular, to retain as much information as possible in $\mathbf{U}$, we propose to reparameterize $\hat{\mathcal{T}}$ and $\mathbf{U}$ into a 'preserved part' and a 'discarded part', where the 'preserved part' contains both the previous information in $\hat{\mathcal{T}}$ and $\mathbf{U}$. The 'retained part' contains the information from the previous $\hat{\mathcal{T}}$ and $\mathbf{U}$, so dropping the 'discarded part' at this point will not have a significant impact on the model. Specifically, we reassemble $\hat{\mathcal{T}}$ and $\mathbf{U}$ together (also known as $\mathcal{T}$) for training, and add some constraints to the filters in $\mathbf{U}$. Finally, we gradually apply more constraints so that the filters in 'the discarded part' almost have zero impact on the model. As such, the re-parameterization can be treated as an optimization problem:

$$\argmin_{\hat{\mathcal{T}}, \mathbf{U}} \ell_{feature}(y, f(x, \hat{\mathcal{T}}, \lambda \mathbf{U})) \quad (11)$$

Algorithm 2: Pruning Approach

- 1 **Input:** unpruned network $\mathcal{T}$
- 2 **Initialization:** architecture set $\mathbf{M}$, accuracy set $\mathbf{Y}$, target pruning rate $\hat{r}$, pruning rate r , iteration t and max iteration t_{max}
- 3 **Output:** pruned network $\hat{\mathcal{T}}_L$
 - 1: **while** $r > \hat{r}$ **and** $t < t_{max}$ **do**
 - 2: Sampling the models from $\hat{\mathcal{T}}$ as in Section 3.2 and validating them for obtaining the $\mathbf{M}$ and $\mathbf{Y}$ pairs.
 - 3: Estimate the hyperparameter $\hat{\rho}_{\eta}^t$ of Bayesian model via Eq.5
 - 4: Re-parameterize the optimal pruned network $\hat{\mathcal{T}}_L$ that obtained by Eq.4 via Eq.11.
 - 5: Fine-tune the pruned network $\hat{\mathcal{T}}_L$.
 - 6: Calculate pruning rate r and **reset** $\hat{\mathcal{T}} = \hat{\mathcal{T}}_L$.
 - 7: **end while**

where

$$\ell_{feature} = d_i + d_j + d_k = \sum_{t=i,j,k} \left\| f^t(x, \mathcal{T}) - f^t(x, \hat{\mathcal{T}} + \lambda \mathbf{U}) \right\|_F^2 \quad (12)$$

According to the block-wise knowledge distillation [17], we pick three layers' feature maps to fulfill the goal of reparameterization. Such a process can also be represented with the dashed box on the right in Figure 3.

By introducing the parameter of λ that represents the probability of remaining it in the pruning process, we call this new step re-parameterization. Such a procedure can make full use of the unpruned model. At this stage, we use the feature maps of the unpruned model as labels and make the pruned model as close as to the unpruned model in terms of these feature maps to reduce the loss from pruning. From Eq.12, we can see that when $\lambda = 0$, if $\ell_{feature}$ is small enough, then $\mathcal{T}$ and $\hat{\mathcal{T}}$ should have the same distribution at this time, which means that optimal pruning is achieved. Note that during the re-parameterization, we fix the learning rate to 0.001 and let λ gradually decreases from 0.5 to 0 in our experiments.

3.5 Bayesian based Pruning Approach

After elucidating the feature representation function-based pruning method and the reparameterization method, our Bayesian modeling-based multi-round pruning method can be mainly represented in Algorithm 2. At each iteration round t , we first sample a number of models and their filter index-accuracy rates by the pruning algorithm in Section 3.3 and use them for Bayesian hyperparameter estimation (red arrows in Figure 1), and then obtain the optimal model by Bayesian prediction and reparameterize it, and of course, Bayesian prediction is also used in the sampling process for the next pruning round to gradually reduce the number of samples (Related analysis is presented in Section 4.2.3).

4 EXPERIMENTS

4.1 Experimental Settings

We validate our method on three mainstream benchmark datasets: CIFAR-10, CIFAR-100 and ImageNet[15]. We also applied Tiny-Imagenet

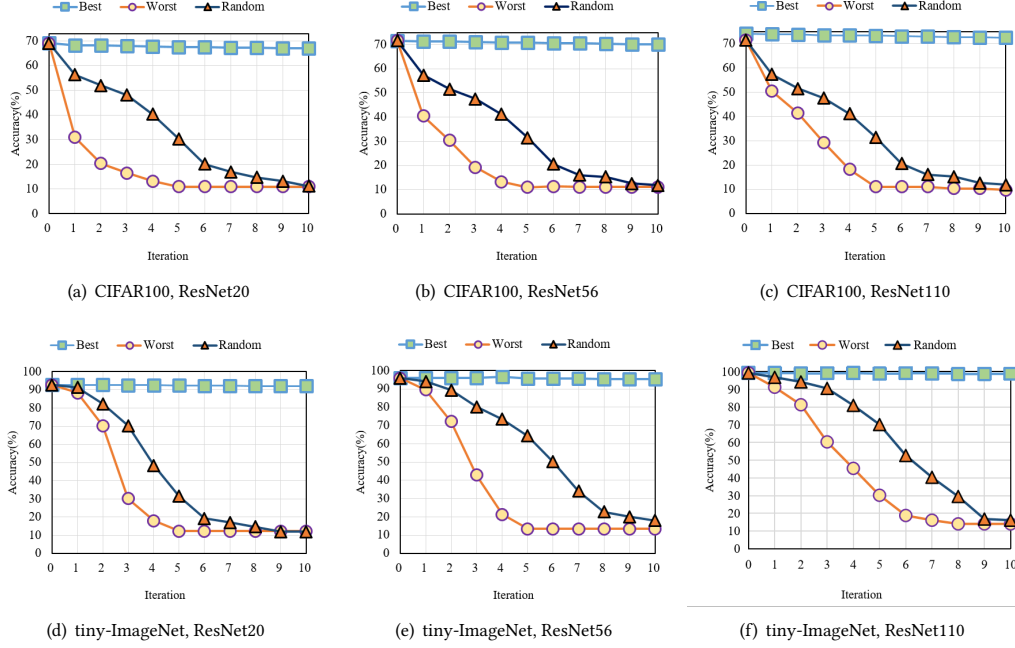


Figure 4: The impact of pruning strategies based on feature distribution functions. On the CIFAR dataset, it can be seen that pruning immediately leads to an irredeemable decrease in accuracy under practices that violate our pruning strategy. Even on the tiny-ImageNet dataset, all but the ‘Best’ two groups lose accuracy after more iterations.

[16] for fast validation, which contains 200 categories with 500 training images, 50 validation images, and 50 test images per category.

We validate two sets of typical models, ResNet[10] and MobileNet V2[33]. For the experiments on CIFAR, we use SGDs with momentum of 0.9 and weight decay of 0.0005 for warmup on a single GPU with batch size of 512. Each model is trained for 400 epochs, and the learning rate is gradually reduced from 0.25 by a cosine scheduler. For ImageNet, we use Adam with weight decay of 0.0001 and train 150 epochs using 4 GPUs with batch size of 256.

After warmup, we prune on the pretrained model to get the pruned model. The details of the pruned model method are described in Sections 3.3 and 3.4. When fine-tuning the pruned models, we take most hyper-parameters as in warmup except learning rate and epoch. For iterative fine-tuning, we set the initial learning rate to 1e-3 and reduce it to 0 by a cosine scheduler, and we train 20/10 epochs per iteration for the models on CIFAR or ImageNet.

4.2 Ablation Studies

4.2.1 Pruning strategy based on feature distribution function. Here, we evaluate the impact of pruning strategies based on feature distribution functions. We use ResNet20/56/110 as unpruned networks and conduct validation experiments with different pruning strategies on CIFAR-100 and Tiny-ImageNet, respectively. We performed 10 iterations to prune the unpruned network by about 50%, and to exclude the effects of randomly initialized parameters, we repeated each set of experiments 5000 times and calculated the mean values as statistics. In each group of experiments, we set up three sets of

control experiments: ‘Best’, ‘Worst’ and ‘Random’ pruning, representing the proposed pruning strategy, the control pruning strategy and random pruning, respectively.

The results shown in Figure 4 indicate that our proposed feature distribution function is sensitive to filter pruning strategy since the corresponding accuracy shows great variations for different pruning strategies. In ‘Best’ pruning, the accuracy loss from each pruning is extremely small through re-parameterization and fine-tuning, and the desired results are eventually obtained, however, the accuracy loss from multiple rounds of pruning in ‘Worst’ pruning and ‘Random’ pruning is difficult to recover, and thus the collapse of accuracy may occur quickly or even in the first few iterations.

4.2.2 Impact of Re-parameterization. We explored the effect of post-pruning reparameterization. Previous pruning rules often perform fine-tuning or knowledge distillation after pruning to recover the accuracy damage caused by pruning, and we deliberately combine fine-tuning and knowledge distillation with our re-parameterization to compare the effects of our re-parameterization. The results are shown in Figure 6. We can see that without the reparameterization, although the accuracy can be maintained in the first few iterations by fine-tuning, however, as the iterative pruning proceeds, the accuracy of the network is difficult to be maintained as in the control group with the addition of reparameterization. Therefore, we believe that re-parameterization based on the probability distribution can lead to a more effective fine-tuning.

4.2.3 Impact of Bayesian estimation. In order to verify the necessity of Bayesian estimation, we designed best validation experiments as described in the above ablation study. We performed 5000 prunings

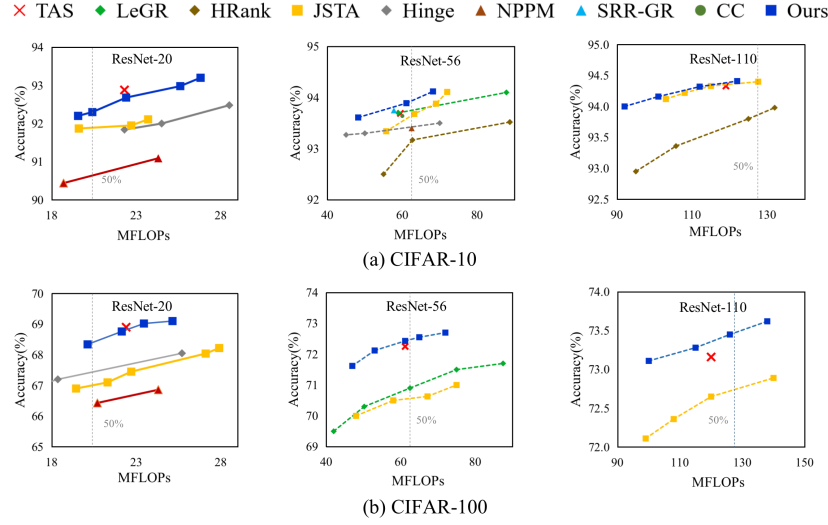


Figure 5: Comparisons with other methods on CIFAR-10 and CIFAR-100, respectively.

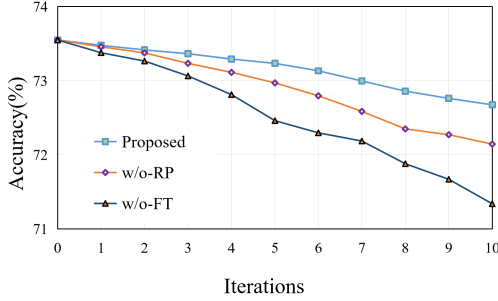


Figure 6: How much parameterization and fine-tuning affects accuracy during iterative pruning.

on different datasets with different models and sampled some of the results in two consecutive rounds of prunings to explore the effect of Bayesian estimation on different rounds of prunings. All the experimental results are consistent, where some of the result data are presented in Table.1. We can see that the performance of pruning without using prior and posterior information (second row) is worse than that of using small samples sampled as posterior (third row), which are both worse than that of using both prior and posterior (fourth row). This proves that the use of Bayesian estimation does speed up the iterative pruning process considerably when sufficient prior is available, and also shows that the each round of pruning is influenced by the previous rounds and also by the subsequent rounds, so that Bayesian estimation can obtain a positive gain. In addition, we can also see that our method can achieve the same level of accuracy with about 20 times' speed-up of sampling.

We further illustrate the accelerating effects of a prior information in 4 consecutive pruning rounds in the Table.2. From the table, we can get the following conclusions:

- *We can save a lot of time when using the prior:* In Table.2 we have utilized number of iterations to measure the total cost. Specifically, we sample 0/10/200/10 networks respectively in

Table 1: Impact of Bayesian estimation. The last four lines represent different sampling and prior conditions.

Testing iteration	With prior	Samples	Testing error
first iter	✓	300	0.8295
next iter	✓	0	1.1154
next iter	×	20	2.8086
next iter	×	200	0.8248
next iter	✓	10	0.8238

the second iteration round, which are divided into 0/10/20/10 networks using our method. As can be seen from the table, pruning using the previous round as a prior is more efficient than pruning only considering the current round (for example, only need to sample 10 groups to The accuracy obtained by sampling 200 groups by other methods can be achieved). Therefore, this set of ablation experiments directly indicates that the same pruning accuracy can be obtained with less computation by using different rounds as priors.

- *With a prior, only a small amount of a posterior information is needed to improve the performance:* comparing the first and fourth groups of each round, it can be found that when providing a prior, if there is not a posterior information (without sample) then the pruning error will gradually become larger as the number of rounds increases, while we only need a small amount of a posterior information (10 samples) to significantly improve the performance of the current round.
- *Using Bayesian-based pruning method not only improves efficiency, but also ensures performance:* in the traditional method, the pruning error gradually increases as the number of pruning rounds increases (from 0.8295 in the first round to 2.2137 in the eighth round), while when we apply Bayesian-based prior and posterior information to pruning, the increase in pruning error is manageable (from 0.8295 in the first round to 0.8545 in the fourth round)

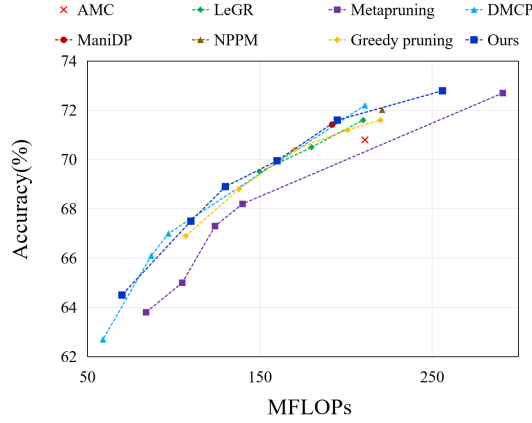


Figure 7: Comparisons on MobileNet V2.

Table 2: Impact of Bayesian estimation (1st-4th iteration). We use the results of the first pruning round in next consecutive four pruning rounds as the prior and verify the effect of the prior information on the subsequent pruning rounds.

Testing iteration	With prior	Samples	Testing error
First iter	✓	300	0.8295
Second iter	✓	0	1.1154
	×	20	2.8086
	×	200	0.8248
	✓	10	0.8238
Third iter	✓	0	1.3273
	×	20	1.6470
	×	200	0.7920
	✓	10	0.7802
Fourth iter	✓	0	1.5547
	×	20	3.7440
	×	180	0.8523
	✓	10	0.8545

4.3 Comparisons with State-of-the-art Approaches

We compare our method with many State-of-the-art (SOTA) pruning methods, including the soft pruning method (SFP)[12], the geometric medium based filter pruning method (FPGM)[13] and the transformable architecture Search (TAS)[5], a joint search-and-training method (JSTA)[26], a differentiable markov pruning method (DMCP)[9], pruning method via learned global ranking (LeGR)[4] filter pruning method based on high order feature map (HRank)[22], a hinge method between pruning and decomposition (Hinge)[19], NPPM [8], SRR-GR[34], and CC [20]. Since some of methods cannot be reproduced, for the sake of fair comparison, we directly use the results reported in the paper as the comparison.

Table 3: Comparisons on ResNet50 with the ImageNet.

Method	FLOPs/Pruned	Acc/Dropped	
		Top-1	Top-5
SFP[12]	2.38e9 / 41.8%	74.61 / 1.54↓	92.06 / 0.81↓
FPGM[13]	2.36e9 / 42.2%	75.50 / 0.65↓	92.63 / 0.21↓
MetaPruning[24]	2.00e9 / 51.3%	75.40 / 1.20↓	-
TAS[5]	2.31e9 / 43.5%	76.20 / 1.26↓	93.07 / 0.48↓
JSTA[26]	1.98e9 / 51.5%	75.21 / 1.45↓	92.56 / 0.54↓
	2.42e9 / 40.8%	75.93 / 0.73↓	92.51 / 0.59↓
HRank[22]	2.30e9 / 43.8%	74.98 / 1.17↓	92.33 / 0.54↓
Hinge[19]	2.30e9 / 46.6%	74.70 / -	-
DMCP[9]	1.10e9 / 73.2%	74.40 / 2.20↓	-
	2.20e9 / 46.1%	76.20 / 0.40↓	-
LeGR[4]	2.37e9 / 42.0%	75.30 / 0.40↓	92.40 / 0.20↓
NPPM[8]	1.81e9 / 56.0%	75.96 / 0.19↓	92.75 / 0.12↓
SRR-GR[34]	1.83e9 / 55.1%	75.11 / 1.02↓	92.35 / 0.51↓
	2.28e9 / 44.1%	75.76 / 0.37↓	92.67 / 0.19↓
CC[20]	1.53e9 / 62.7%	74.54 / 1.61↓	92.25 / 0.62↓
	1.93e9 / 52.9%	75.59 / 0.56↓	92.64 / 0.23↓
Ours	1.97e9 / 52.8%	75.85 / 0.83↓	92.55 / 0.37↓
	2.21e9 / 45.9%	76.13 / 0.55↓	92.68 / 0.26↓
	2.42e9 / 40.8%	76.29 / 0.39↓	92.87 / 0.07↓

In Fig. 5, we show a comparison of parameters, FLOPs, and accuracy on CIFAR datasets with different ResNet models. It is obvious that our method can achieve higher accuracy on two datasets and different models, even under the premise of less FLOPs.

For the ImageNet dataset, we show the comparisons in Table 3 and Fig. 7 for our comparative validation on the ResNet50 and MobileNet V2, respectively. Comparing with existing methods, our approach provides similar or better results. Note that because our method uses Bayesian estimation to accelerate iterative pruning, it takes much less time to achieve the demonstrated pruning results than other iterative pruning methods.

5 CONCLUSIONS

In this work, we have introduced a novel Bayesian based reparameterization model pruning approach. The channel importance is represented by the distribution discrepancy of feature maps based on Bayesian. To further reduce the performance gap between the pruned models and original ones, a novel reparameterization step is introduced. Extensive ablation study and comparison experiments on benchmark datasets show that the proposed approach can achieve new SOTA performance for DNN model pruning.

ACKNOWLEDGMENTS

This work was supported in part by the National Key RD Program of China under Grant 2018AAA0101400 and the Natural Science Foundation of China under Grant 61991451, Grant 61632019, Grant 61621005, and Grant 61836008.

REFERENCES

- [1] Han Cai, Chuang Gan, Tianze Wang, Zhekai Zhang, and Song Han. 2019. Once-for-All: Train One Network and Specialize it for Efficient Deployment.
- [2] Han Cai, Ligeng Zhu, and Song Han. 2018. Proxylessnas: Direct neural architecture search on target task and hardware. (2018).
- [3] Pengguang Chen, Shu Liu, Hengshuang Zhao, and Jiaya Jia. 2021. Distilling Knowledge via Knowledge Review. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 5008–5017.
- [4] Ting-Wu Chin, Ruizhou Ding, Cha Zhang, and Diana Marculescu. 2020. Towards Efficient Model Compression via Learned Global Ranking. 1518–1528.
- [5] Xuanyi Dong and Yi Yang. 2019. Network pruning via transformable architecture search. 759–770.
- [6] Richard M Dudley. 2010. Sample functions of the Gaussian process. *Selected Works of RM Dudley* (2010), 187–224.
- [7] R. Eldan and O. Shamir. 2015. The Power of Depth for Feedforward Neural Networks. *Computer Science* (2015).
- [8] Shangqian Gao, Feihu Huang, Weidong Cai, and Heng Huang. 2021. Network Pruning via Performance Maximization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 9270–9280.
- [9] Shaopeng Guo, Yujie Wang, Quanquan Li, and Junjie Yan. 2020. DMCP: Differentiable Markov Channel Pruning for Neural Networks. 1539–1547.
- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun, and Ieee. 2016. Deep Residual Learning for Image Recognition (*IEEE Conference on Computer Vision and Pattern Recognition*). 770–778. <https://doi.org/10.1109/cvpr.2016.90>
- [11] Yang He, Yuhang Ding, Ping Liu, Linchao Zhu, Hanwang Zhang, and Yi Yang. 2020. Learning Filter Pruning Criteria for Deep Convolutional Neural Networks Acceleration. 2009–2018.
- [12] Yang He, Guoliang Kang, Xuanyi Dong, Yanwei Fu, and Yi Yang. 2018. Soft filter pruning for accelerating deep convolutional neural networks.
- [13] Yang He, Ping Liu, Ziwei Wang, Zhilan Hu, and Yi Yang. 2019. Filter pruning via geometric median for deep convolutional neural networks acceleration. 4340–4349.
- [14] Tao Huang, Weisheng Dong, Jinshan Liu, Fangfang Wu, Guang ming Shi, and Xin Li. 2020. Accelerating Convolutional Neural Network via Structured Gaussian Scale Mixture Models: a Joint Grouping and Pruning Approach. (2020).
- [15] Alex Krizhevsky, I. Sutskever, and G. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks.
- [16] Ya Le and Xuan Yang. 2015. Tiny imagenet visual recognition challenge. *CS 231N* 7 (2015), 7.
- [17] Changlin Li, Jiefeng Peng, Liuchun Yuan, Guangrun Wang, Xiaodan Liang, Liang Lin, and Xiaojun Chang. 2020. Blockwisely Supervised Neural Architecture Search with Knowledge Distillation. *arXiv:1911.13053 [cs.CV]*
- [18] Hao Li, Asim Kadav, Igor Durdanovic, Hanan Samet, and Hans Peter Graf. 2016. Pruning filters for efficient convnets. *arXiv preprint arXiv:1608.08710* (2016).
- [19] Yawei Li, Shuhang Gu, Christoph Mayer, Luc Van Gool, and Radu Timofte. 2020. Group sparsity: The hinge between filter pruning and decomposition for network compression. 8018–8027.
- [20] Yuchao Li, Shaohui Lin, Jianzhuang Liu, Qixiang Ye, Mengdi Wang, Fei Chao, Fan Yang, Jincheng Ma, Qi Tian, and Rongrong Ji. 2021. Towards Compact CNNs via Collaborative Compression. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6438–6447.
- [21] Zhihang Li, Teng Xi, Jiankang Deng, Gang Zhang, Shengzhao Wen, and Ran He. 2020. Gp-nas: Gaussian process based neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 11933–11942.
- [22] Mingbao Lin, Rongrong Ji, Yan Wang, Yichen Zhang, Baochang Zhang, Yonghong Tian, and Ling Shao. 2020. HRank: Filter Pruning using High-Rank Feature Map. 1529–1538.
- [23] Hanxiao Liu, Karen Simonyan, and Yiming Yang. 2018. Darts: Differentiable architecture search. (2018).
- [24] Zechun Liu, Haoyuan Mu, Xiangyu Zhang, Zichao Guo, Xin Yang, Kwang-Ting Cheng, and Jian Sun. 2019. Metapruning: Meta learning for automatic neural network channel pruning. 3296–3305.
- [25] Christos Louizos, Karen Ullrich, and Max Welling. 2017. Bayesian compression for deep learning. *Advances in neural information processing systems* 30 (2017).
- [26] Xiaotong Lu, Han Huang, Weisheng Dong, Xin Li, and Guangming Shi. 2020. Beyond network pruning: a joint search-and-training approach.
- [27] Chao Ma, Jia-Bin Huang, Xiaokang Yang, Ming-Hsuan Yang, and Ieee. 2015. Hierarchical Convolutional Features for Visual Tracking (*IEEE International Conference on Computer Vision*). 3074–3082. <https://doi.org/10.1109/iccv.2015.352>
- [28] Kirill Neklyudov, Dmitry Molchanov, Arsenii Ashukha, and Dmitry P Vetrov. 2017. Structured bayesian pruning via log-normal multiplicative noise. *Advances in Neural Information Processing Systems* 30 (2017).
- [29] Hieu Pham, Melody Y Guan, Barret Zoph, Quoc V Le, and Jeff Dean. 2018. Efficient neural architecture search via parameter sharing. (2018).
- [30] Carl Edward Rasmussen. 2003. Gaussian processes for machine learning.
- [31] Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. 2019. Regularized evolution for image classifier architecture search, Vol. 33. 4780–4789.
- [32] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2017. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. 39, 6 (2017), 1137–1149. [GotoISI>://WOS:000401091200007](https://arxiv.org/abs/1601.01424)
- [33] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. 4510–4520.
- [34] Zi Wang, Chengcheng Li, and Xiangyang Wang. 2021. Convolutional neural network pruning with structural redundancy reduction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14913–14922.
- [35] Xin Yuan, Liangliang Ren, Jiwen Lu, and Jie Zhou. 2019. Enhanced bayesian compression via deep reinforcement learning. 6946–6955.
- [36] Yuefu Zhou, Ya Zhang, Yanfeng Wang, and Qi Tian. 2019. Accelerate cnn via recursive bayesian pruning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 3306–3315.
- [37] Barret Zoph and Quoc V Le. 2016. Neural architecture search with reinforcement learning. (2016).